

FCI – Faculdade de Computação e Informática

CompPar: Laboratório Pthreads Programação Concorrente em Memória Compartilhada e Análise do GIL

Ciência da Computação - Disciplina de Computação Paralela

Nome: Kauê Lima Rodrigues Meneses - **RA:** 10410594

Nome: Theo Esposito Simões Resende - **RA:** 10721356

Repositório:

<https://github.com/Kaue557/Computacao-Paralela/tree/main/Lab03-CompPar>

1. Introdução e Fundamentos Teóricos

Paralelismo (Pthreads): Execução simultânea de threads em **múltiplos núcleos**, rodando dentro do **mesmo processo** e compartilhando a mesma memória (RAM e variáveis).

Condições de Corrida: Falhas e perda de dados que ocorrem quando múltiplas threads tentam modificar variáveis compartilhadas ao **mesmo tempo**, atropelando as instruções umas das outras.

Sincronização (Mutex): Uma trava (exclusão mútua) que protege variáveis compartilhadas, forçando as threads a acessarem o recurso uma por vez. Isso garante o resultado, mas causa perda de desempenho (overhead).

Divisão de Trabalho: Estratégias para distribuir o processamento: pode ser em blocos contíguos (otimiza o uso da memória cache) ou cíclica (garante que threads terminem juntas, balanceando a carga).

GIL (Python): É uma trava do interpretador CPython que impede o paralelismo real. Ela força threads de processamento pesado (CPU-bound) a executarem sequencialmente, muitas vezes deixando o código concorrente **mais lento que o sequencial**.

2. Metodologia e Ambiente Experimental

Os ensaios foram conduzidos numa infraestrutura com as seguintes especificações:

1. **Processador:**

- Modelo: Ryzen 7 7734HS
- Frequência base: 3.2 GHz
- Frequência Turbo máxima: 4,75 GHz
- Núcleos físicos: 8
- Núcleos lógicos (threads): 16

2. **Hierarquia de memória cache:**

- L1: 512 KB
- L2: 4 MB
- L3: 16 MB

3. **Memória RAM:** 16GB DDR5

4. **Sistema Operacional:**

- Distribuição: Fedora KDE 44
- Versão do kernel Linux: 7.2.4-200.fc44.x86_64

5. **Compilador e Interpretador** (versões do gcc, glibc e python3, acompanhadas das flags exatas de compilação, e.g., -O2 -Wall -pthread)

- GCC: 16.2.1 20260819 (Red Hat 16.2.1-2)
- glibc 2.43
- Python 3.14.7

3. Evidências Experimentais - Capturas de Tela

Parte 1: O Problema da Conta Corrente

```
thesposito@fedora:~/Projects/Computacao-Paralela/Lab03-CompPar$ ./run_evidencias.sh
=====
PARTE 1 - FASE 0: contacorrente_seq.c
=====
COMPILANDO: gcc -O2 -Wall contacorrente_seq.c -o contacorrente_seq
EXECUTANDO: ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.0656 s
```

Fase 0 - Sequencial

```
=====
PARTE 1 - FASE 1: contacorrente_f1.c
=====
COMPILANDO: gcc -O2 -Wall contacorrente_f1.c -o contacorrente_f1 -pthread
EXECUTANDO 5 VEZES: ./contacorrente_f1
--- Execução 1 ---
[Fase 1 - Race Condition] Saldo final: -99999000.00 | Tempo: 0.0337 s
--- Execução 2 ---
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0337 s
--- Execução 3 ---
[Fase 1 - Race Condition] Saldo final: -99999000.00 | Tempo: 0.0337 s
--- Execução 4 ---
[Fase 1 - Race Condition] Saldo final: -99999000.00 | Tempo: 0.0342 s
--- Execução 5 ---
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0342 s
```

Fase 1 - Condição de Corrida

```
=====
PARTE 1 - FASE 2: contacorrente_f2.c
=====
COMPILANDO: gcc -O2 -Wall contacorrente_f2.c -o contacorrente_f2 -pthread
EXECUTANDO 5 VEZES: ./contacorrente_f2
--- Execução 1 ---
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 1.9587 s
--- Execução 2 ---
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 2.2818 s
--- Execução 3 ---
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 2.1977 s
--- Execução 4 ---
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 2.1218 s
--- Execução 5 ---
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 1.9418 s
```

Fase 2 - Correção com Mutex

Parte 2: Menu de Problemas

```
=====
PARTE 2 - OPÇÃO A: primos.c (K=5000000)
=====
COMPILANDO: gcc -O2 -Wall primos.c -o primos -pthread
EXECUTANDO: ./primos 5000000 1
Total de primos: 348513
Tempo: 0.520591 segundos
EXECUTANDO: ./primos 5000000 2
Total de primos: 348513
Tempo: 0.519019 segundos
EXECUTANDO: ./primos 5000000 4
Total de primos: 348513
Tempo: 0.261211 segundos
EXECUTANDO: ./primos 5000000 8
Total de primos: 348513
Tempo: 0.132672 segundos
```

Opção A: Busca e Contagem de Números Primos

```
=====
PARTE 2 - OPÇÃO C: soma_vetores.c (N=50000000)
=====
COMPILANDO: gcc -O2 -Wall soma_vetores.c -o soma_vetores -pthread
EXECUTANDO: ./soma_vetores 50000000 1
Soma concluída. Amostra C[N-1]: 100000000.00
Tempo: 0.108795 segundos
EXECUTANDO: ./soma_vetores 50000000 2
Soma concluída. Amostra C[N-1]: 100000000.00
Tempo: 0.057730 segundos
EXECUTANDO: ./soma_vetores 50000000 4
Soma concluída. Amostra C[N-1]: 100000000.00
Tempo: 0.033626 segundos
EXECUTANDO: ./soma_vetores 50000000 8
Soma concluída. Amostra C[N-1]: 100000000.00
Tempo: 0.026907 segundos
```

Opção C: Soma de Vetores Gigantes

```

=====
PARTE 2 - OPÇÃO E: matriz_vetor.c (N=10000)
=====
COMPILANDO: gcc -O2 -Wall matriz_vetor.c -o matriz_vetor -pthread
EXECUTANDO: ./matriz_vetor 10000 1
Multiplicação concluída. Amostra R[N-1]: 149985000.00
Tempo: 0.080450 segundos
EXECUTANDO: ./matriz_vetor 10000 2
Multiplicação concluída. Amostra R[N-1]: 149985000.00
Tempo: 0.041974 segundos
EXECUTANDO: ./matriz_vetor 10000 4
Multiplicação concluída. Amostra R[N-1]: 149985000.00
Tempo: 0.023489 segundos
EXECUTANDO: ./matriz_vetor 10000 8
Multiplicação concluída. Amostra R[N-1]: 149985000.00
Tempo: 0.020969 segundos

```

Opção E: Multiplicação de Matriz por Vetor

Parte 3: O Desafio do Python e o GIL

```

=====
PARTE 3: seq.py
=====
EXECUTANDO: python3 seq.py
[Python Sequencial] Tempo: 4.3839 segundos

=====
PARTE 3: threads.py
=====
EXECUTANDO: python3 threads.py
[Python com Threads] Tempo: 4.5037 segundos

=====
FIM DA EXECUÇÃO
=====
thesposito@fedora:~/Projects/Computacao-Paralela/Lab03-CompPar$

```

4. Tabela Comparativa de Desempenho e Gráficos de Speedup

Tempos médios de execução

Programa	Parâmetros	Threads	Tempo Médio (s)
contacorrente_seq	N/A	1	69.415
contacorrente_f1	N/A	2	37.559
contacorrente_f2	N/A	2	2.054.107
primos	5000000	1	526.818
primos	5000000	2	528.900
primos	5000000	4	270.156
primos	5000000	8	141.425
soma_vetores	50000000	1	376.684
soma_vetores	50000000	2	329.775
soma_vetores	50000000	4	305.666
soma_vetores	50000000	8	299.481
matriz_vetor	10000	1	567.183
matriz_vetor	10000	2	505.799
matriz_vetor	10000	4	481.287
matriz_vetor	10000	8	466.074
seq.py	N/A	1	4.478.534
threads.py	N/A	2	4.758.140

Cálculo de Speedup e Eficiência

Programa	Parâmetros	Threads	Tempo Médio (s)	SpeedUp	Eficiência
primos	5000000	1	526.818	1.2000	1.2000
primos	5000000	2	528.900	9.961	4.980
primos	5000000	4	270.156	1.9501	4.875
primos	5000000	8	141.425	3.7251	4.656
soma_vetores	50000000	1	376.684	1.2000	1.2000
soma_vetores	50000000	2	329.775	1.1422	5.711
soma_vetores	50000000	4	305.666	1.2323	3.081
soma_vetores	50000000	8	299.481	1.2578	1.572
matriz_vetor	10000	1	567.183	1.2000	1.2000
matriz_vetor	10000	2	505.799	1.1214	5.607
matriz_vetor	10000	4	481.287	1.1785	2.946
matriz_vetor	10000	8	466.074	1.2169	1.521

5. Questões de Reflexão

1. Ciclo de Leitura-Modificação-Escrita (Parte 1 - Fase 1): Explique, em nível de instruções de máquina (assembly) e hierarquia de memória (registradores, cache L1 e RAM), por que a operação `saldo += VALOR_DEPOSITO` não é atômica e como isso provoca a perda sistemática de atualizações (lost updates) na Fase 1.

R:

A instrução `saldo += VALOR_DEPOSITO` não é atômica porque, a **nível de processamento**, desdobra-se em **três instruções** de linguagem máquina distintas (ciclo Leitura-Modificação-Escrita):

Leitura (Load): O valor da variável partilhada `saldo` é lido da hierarquia de memória (RAM ou Cache L1) e carregado para um registrador privado do processador.

Modificação (Add): O processador soma o `VALOR_DEPOSITO` ao valor presente unicamente nesse registrador local.

Escrita (Store): O resultado final contido no registrador é transferido de volta para a memória partilhada (Cache L1/RAM), atualizando a variável. Como ocorre a perda de atualizações (lost updates):

Como os registradores são **memórias isoladas** dentro de cada núcleo do processador, se a Thread A e a Thread B executarem a fase de "Leitura" no mesmo momento, ambas vão carregar exatamente o mesmo `saldo` original da memória compartilhada. Ambas realizam a soma de forma **isolada** e efetuam a "Escrita" de forma **consecutiva**. Consequentemente, o valor que for guardado na memória em último lugar vai sobrepor o valor gravado pela outra thread, fazendo com que uma das operações seja perdida.

2. Impacto e Granularidade do Mutex (Parte 1 - Fase 2): Como o mecanismo `pthread_mutex_t` restabelece a consistência matemática dos dados? Analise o impacto no tempo de execução comparando a Fase 1 e a Fase 2, explicando o custo computacional associado à troca de contexto e contenção de locks quando inserido no interior de um laço de milhões de iterações.

R:

O mecanismo `pthread_mutex_t` restabelece a consistência matemática porque atua como uma "trava de exclusão" mútua em torno da região

crítica do código (a instrução de atualização do saldo, nesse caso). Ele garante que apenas **uma thread por vez** possa executar o ciclo de leitura-modificação-escrita, impedindo que as threads se atropelam e gerem condições de corrida.

Em relação ao impacto no tempo de execução, a Fase 2 (com Mutex) torna-se substancialmente mais lenta que a Fase 1. Isso ocorre devido ao extremo custo computacional introduzido pela granularidade fina do lock:

Contenção de Locks (Lock Contention): Inserir um mutex dentro de um laço com milhões de iterações faz com que as threads passem a maior parte do tempo "brigando" para adquirir a trava, em vez de processar o cálculo.

Troca de Contexto (Context Switch): Quando uma thread tenta pegar o mutex e ele já está ocupado, o sistema operacional suspende essa thread e acorda a outra. Esse revezamento contínuo gasta muitos ciclos de processador apenas no gerenciamento do sistema, gerando uma enorme sobrecarga (overhead) e destruindo o ganho de desempenho do paralelismo.

3. Estratégias Alternativas de Sincronização (Parte 1): Seria possível resolver o problema da conta corrente sem introduzir 100 milhões de operações de lock? Proponha e descreva conceitualmente duas alternativas: (a) usando variáveis locais por thread com redução única final; e (b) utilizando operações atômicas nativas (ex.: C11).

R:

Sim, é possível.

Alternativa (a): Variáveis locais com redução única final: Em vez de acessar a variável global a cada repetição, cada thread declara uma variável privada (ex: saldo_local) e realiza as 50 milhões de operações (saques ou depósitos) inteiramente nela, sem precisar de nenhum mutex. Ao sair do laço, a thread adquire o mutex uma única vez e soma o seu resultado final na variável global compartilhada da conta corrente. Isso reduz o número de locks de 100 milhões para apenas dois (um por thread), eliminando essa contenção.

Alternativa (b): Operações atômicas nativas (C11): Utilizando a biblioteca <stdatomic.h>, o tipo da variável global é declarado como atômico (ex: `_Atomic double`). O compilador traduz a operação matemática para instruções especiais de hardware que travam o acesso ao nível do barramento de memória ou cache do processador durante o ciclo de leitura-modificação-escrita. Isso delega a exclusão mútua diretamente para o processador de forma ininterrupta, garantindo o resultado correto sem utilizar a sobrecarga do sistema operacional (como faz o mutex).

4. Decomposição e Balanceamento de Carga (Parte 2): Para os problemas que você escolheu no Menu, descreva qual estratégia de particionamento de trabalho foi empregada (estática por blocos, cíclica ou dinâmica). Como o desbalanceamento de carga afeta a eficiência do paralelismo, especialmente em algoritmos como a busca de números primos?

R:

Estratégias de Particionamento de Trabalho Empregadas:

Soma de Vetores Gigantes e Multiplicação Matriz x Vetor: Foi utilizada a distribuição estática por blocos contíguos. O conjunto total de dados (linhas da matriz ou índices do vetor) foi dividido em pedaços contínuos e uniformes, o que é ideal para operações de custo computacional constante e favorece a localidade espacial na memória cache.

Busca e Contagem de Números Primos: Foi empregada a distribuição cíclica (onde as threads alternam os números saltando de acordo com o total de threads).

Impacto do Desbalanceamento de Carga: O desbalanceamento de carga ocorre quando o trabalho não é distribuído de forma igualitária, fazendo com que o tempo total de execução seja ditado pela thread mais sobrecarregada, enquanto as outras ficam ociosas esperando. Isso afeta diretamente a eficiência e limita a aceleração (*Speedup*) do sistema.

Na busca de números primos, o teste de primalidade não é uniforme: verificar se um número muito grande é primo consome consideravelmente mais ciclos de processador do que testar um número pequeno. Se fosse aplicada uma divisão de blocos estáticos, as threads responsáveis pelo final do intervalo demorariam **muito mais**

tempo do que as iniciais, gerando desbalanceamento. A distribuição cíclica resolve esse problema ao misturar números pequenos e grandes para todas as threads, mantendo-as ocupadas o mesmo tempo.

5. Reentrância e Thread-Safety em Bibliotecas (Parte 2 - Opção B): Por que a função tradicional `rand()` degrada o desempenho de programas multithread? Explique o conceito de função reentrante e diferencie a assinatura e o funcionamento de `rand()` e `rand_r()`.

R:

Degradação do `rand()`: A função tradicional `rand()` degrada o desempenho porque possui um estado global protegido internamente por um lock na biblioteca do C (glibc). Quando várias threads tentam gerar números aleatórios ao mesmo tempo, esse lock bloqueia todas menos uma, serializando a execução e anulando todo o ganho paralelo.

Função Reentrante (Thread-Safe): É uma função que pode ser executada simultaneamente por múltiplas threads de forma totalmente segura, sem causar problemas ou gerar resultados incorretos. Funções reentrantes não armazenam dados em variáveis estáticas globais entre chamadas consecutivas, garantindo que o estado de uma thread não interfira no da outra.

Diferença entre `rand()` e `rand_r()`:

`int rand(void)`: Não recebe parâmetros e baseia-se em uma única semente global compartilhada, necessitando de travamento interno.

`int rand_r(unsigned int *seedp)`: Exige que o estado seja passado como um ponteiro por quem chama a função. Como cada thread cria e fornece a sua própria variável de semente (seed) independente, a função opera apenas com os dados locais daquela thread, eliminando a necessidade de locks e garantindo o paralelismo real.

6. Escalabilidade e Limitações Físicas de Memória (Parte 2 - Opção C/E): Ao variar de 1 para 2, 4 e 8 threads, os problemas atingiram Speedup linear? Em algoritmos limitados por memória (memory-bound, como a soma de vetores gigantes), por que a taxa de transferência do barramento (largura de banda da DRAM) atua como um teto insuperável para a aceleração?

R:

Aceleração Linear (Speedup): Os dados experimentais evidenciam que o Speedup linear não foi atingido em sua totalidade, ocorrendo uma saturação evidente à medida que o número de threads aumentou.

- Na soma de vetores (Opção C), o tempo reduziu quase pela metade ao passar de 1 thread (0.108s) para 2 threads (0.057s), mas a eficiência caiu significativamente ao utilizar 8 threads (0.026s, o que representa um Speedup de aproximadamente 4x, e não 8x).
- Na multiplicação matriz-vetor (Opção E), o comportamento foi similar: a execução passou de 0.080s com 1 thread para 0.020s com 8 threads, resultando novamente em um teto de aceleração na casa de 4x.

O Teto do Barramento de Memória (Memory-Bound):

Algoritmos como a soma de vetores gigantes são classificados como limitados por memória (memory-bound) porque a operação matemática realizada neles é extremamente simples (custo computacional de $O(1)$ por elemento processado).

A largura de banda da memória RAM (DRAM) impõe um limite físico absoluto na quantidade máxima de dados (em bytes) que o barramento da placa-mãe consegue trafegar até o processador por segundo.

Quando **múltiplas threads** solicitam enormes blocos de dados simultaneamente, o barramento satura rapidamente, atingindo a sua taxa máxima de transferência. A partir desse ponto, adicionar novos núcleos computacionais não gera ganho de tempo, pois os processadores ficam ociosos aguardando a chegada dos dados da memória.

7. O Papel do GIL no CPython (Parte 3): Defina o que é o Global Interpreter Lock (GIL), por que ele foi historicamente implementado no interpretador CPython (em termos de gerenciamento de memória e contagem de referências) e por que ele inviabiliza ganhos de desempenho em códigos puramente CPU-bound que utilizam o módulo threading.

R:

O que é o GIL: O Global Interpreter Lock (GIL) é uma trava de exclusão mútua (mutex) interna do interpretador CPython que restringe a

execução de bytecode Python a apenas uma única thread por vez, independentemente de o processador possuir múltiplos núcleos lógicos.

Motivo histórico (Gerenciamento de Memória): O CPython utiliza um mecanismo chamado contagem de referências (reference counting) para gerenciar a memória e o ciclo de vida dos objetos. Se múltiplas threads nativas alterassem esse contador numérico simultaneamente sem sincronização, ocorreriam **condições de corrida** que causariam **vazamento de memória** ou exclusão prematura de variáveis em uso. O GIL foi implementado como uma solução simples e altamente eficiente para o código sequencial, protegendo toda a memória do interpretador de uma só vez por meio de uma trava global única, evitando a necessidade de colocar milhares de locks granulares em cada objeto individualmente.

Inviabilidade em códigos CPU-bound: Em tarefas puramente CPU-bound (como uma contagem decrescente massiva ou cálculos numéricos intensos), as threads exigem processamento contínuo. Como o GIL obriga que apenas uma thread execute instruções por vez, o paralelismo real é **anulado**. Os núcleos adicionais do processador ficam ociosos enquanto o sistema operacional gasta poder de processamento realizando inúmeras trocas de contexto (context switches) para revezar o controle do GIL entre as threads. É por isso que o código concorrente em Python não fica mais rápido e, frequentemente, torna-se ligeiramente mais lento que a versão puramente sequencial.

8. Paralelismo Real em Python (Parte 3): Como o módulo padrão multiprocessing contorna o limite imposto pelo GIL? Quais são as desvantagens e sobrecargas (overheads) do modelo de múltiplos processos com memória isolada quando comparado ao modelo de múltiplas threads com memória compartilhada (ex.: custo de serialização via pickle e comunicação interprocessos - IPC)?

R:

Como o multiprocessing contorna o GIL: O módulo multiprocessing contorna essa limitação **criando novos processos** independentes no sistema operacional, em vez de criar threads dentro do mesmo processo. Como cada novo processo recebe o seu próprio interpretador Python instanciado e o seu próprio espaço de memória isolado, cada um passa a ter o seu próprio GIL exclusivo. Sem a

partilha de memória, não há competição pela mesma trava, permitindo que as tarefas rodem em paralelismo real nos múltiplos núcleos do processador.

Desvantagens e Sobrecargas (Overheads): Embora permita o paralelismo real, o modelo de múltiplos processos com memória isolada apresenta desvantagens significativas em comparação às threads de memória compartilhada:

- **Alto custo de criação:** Instanciar um processo completo exige muito mais alocação de memória e tempo do sistema operacional do que criar uma thread "leve".

- **Comunicação Interprocessos (IPC):** Como os processos não compartilham o mesmo espaço de endereçamento, eles não podem simplesmente ler e escrever na mesma variável global. A troca de informações exige mecanismos de IPC (como pipes, filas ou sockets), que introduzem latência.

- **Serialização (Pickle):** Para transmitir dados entre os processos através de IPC, o Python é obrigado a traduzir os objetos em um fluxo de bytes (serialização via pickle) no envio, e reconstruí-los no destino. Esse empacotamento e desempacotamento contínuo de dados gera um overhead computacional elevado, especialmente ao lidar com grandes volumes de dados (como matrizes e vetores gigantes), algo que não ocorre em Pthreads.

9. Limites Teóricos e Lei de Amdahl: Considerando as frações sequenciais intrínsecas aos programas desenvolvidos (como alocação de memória, inicialização de estruturas, criação de threads com `pthread_create` e junção com `pthread_join`), aplique a Lei de Amdahl para estimar o limite teórico máximo de Speedup que seu sistema poderia atingir, mesmo que dispusesse de um número infinito de núcleos de processamento.

R:

A Lei de Amdahl e a Fração Sequencial: A Lei de Amdahl estabelece que a aceleração (Speedup) máxima de um programa é fundamentalmente limitada pela sua fração estritamente sequencial (f), ou seja, a parte do código que não pode ser paralelizada. Nos programas desenvolvidos, operações estruturais como `malloc` (alocação), o preenchimento de dados pelo `main`, a chamada de `pthread_create` e a barreira de espera `pthread_join` representam essa fração sequencial que obrigatoriamente roda em apenas um núcleo.

Aplicação do Limite Teórico (Núcleos Infinitos): A fórmula matemática da Lei de Amdahl é definida como:

$S = 1/(f + (1-f)/N)$. Onde f é a fração sequencial, $(1-f)$ é a fração paralelizável e N é o número de núcleos (ou threads).

Se considerarmos um sistema hipotético com um número infinito de núcleos de processamento ($N \rightarrow \infty$), o tempo necessário para executar a parte paralela $((1-f)/N)$ tenderá a zero. A equação então se simplifica para o limite teórico máximo insuperável: $\text{SpeedupMax} = 1/f$

Estimativa Prática: Se a criação das threads, alocação de memória e impressão de resultados consumirem, por exemplo, 5% do tempo total original do seu programa sequencial ($f = 0.05$), o cálculo será: $\text{SpeedupMax} = 1/0.05 = 20$

Isso significa que, mesmo que você utilize um supercomputador com infinitos núcleos para processar o laço paralelo instantaneamente, o seu programa jamais será mais do que 20 vezes mais rápido do que a versão original de 1 thread, porque o gargalo sequencial de 5% atua como um teto físico para a escalabilidade.

6. Dificuldades Técnicas e Soluções

Relato transparente dos obstáculos encontrados durante a implementação (por exemplo: passagem e conversão de parâmetros em ponteiros genéricos void, contenção de mutexes, balanceamento de trabalho, garantia de aleatoriedade independente com `rand_r`, gerenciamento de cache e memória) e a metodologia adotada pela equipe para superá-los.

R:

Tivemos dificuldades em relação aos códigos escolhidos do Menu principalmente sobre uso correto de **ponteiros** e seu entendimento, **tipagem** precisa de variáveis e **alocação dinâmica de memória**. Usamos ferramentas de **inteligência artificial** (copilot, por exemplo) para **corrigir** tais erros e explicar o motivo deles, assim compreendendo de fato a sintaxe e estrutura das soluções.

7. Declaração e Análise do Uso de IA:

Mediante o uso de Inteligência Artificial, a mesma foi proveitosa e crucial para a precisão e dinamismo do desenvolvimento do relatório, desde esclarecimento de conceitos até edição/debugging direto nos códigos. Acerca da veracidade, nenhuma das respostas e códigos de devolutiva pareceram estar imprecisos em relação a matéria e regras apresentadas em aula.

O aprendizado se tornou mais claro e palpável, já que tivemos a ajuda de uma espécie de “professor particular” para nos auxiliar; sempre mantendo, claro, o cuidado e atenção para um uso saudável.

Olá, Claude! Quero que você me ajude com o tópico 3 do relatório:

3. Evidências Experimentais (Capturas de Tela Obrigatórias):

- Screenshots nítidos do terminal comprovando a compilação limpa e a execução da Parte 1 (Fase 0 - Sequencial, Fase 1 - Condição de Corrida evidenciando a flutuação e corrupção do saldo em execuções repetidas, e Fase 2 - Correção com Mutex comprovando o saldo correto determinístico).
- Screenshots da compilação e execução dos 3 problemas selecionados da Parte 2 (Menu de Problemas) executados para 1, 2, 4 e 8 threads, comprovando a exatidão das saídas geradas.
- Screenshots da execução comparativa da Parte 3 no terminal exibindo o tempo obtido em seq.py versus threads.py.

Para isso, quero que redija um script para execução de cada um dos arquivos referentes a esta etapa com as flags de compilação mencionadas. O script não deve exibir nenhum comentário ou reflexão. Deixe sempre claro qual arquivo está sendo executado e quais parâmetros. Segue lista com os nomes dos arquivos:

thesposito@fedora:~/Projects/Computacao-Paralela/Lab03-CompPar\$ tree .

```
.
├── contacorrente_f1.c
├── contacorrente_f2.c
├── contacorrente_seq.c
├── matriz_vetor.c
├── primos.c
├── seq.py
├── soma_vetores.c
└── threads.py
```

1 directory, 8 files

Mostrar menos

Automação de testes - Claude

Okay! Só pra documentar, renomeei o arquivo para "run_evidencias.sh" e incluí no repositório. Fiz algumas pequenas alterações porque não achei necessário incluir logs.

Agora me ajude com o tópico 4 do relatório:

"4. Tabela Comparativa de Desempenho e Gráficos de Speedup:

- Tabelas estruturadas contendo os tempos médios de execução de todas as etapas práticas (mínimo de 3 a 5 execuções por medição para obtenção da média aritmética).
- Preenchimento da tabela de escalabilidade para as 3 opções do Menu variando de 1 a 8 threads.
- Cálculo formal do Speedup ($Sp = T1/Tp$) e da Eficiência ($Ep = Sp/p$).
- Gráficos comparativos de Speedup × Número de Threads ou análise discursiva aprofundada justificando o comportamento de aceleração ou saturação observado."

Crie um script chamado "medir.sh" para fazer essas medições, exibindo os resultados via terminal e depois salvando os dados necessários em um arquivo csv. Use os mesmos parâmetros e argumentos (use as flags corretas) que utilizou no script anterior.

Mostrar menos

há 29 minutos 

Medições e estatísticas - Claude

o que é um "dunder method" em python?
esse, por exemplo:
import time

```
def contagem_pesada(n):  
    while n > 0:  
        n -= 1
```

```
if __name__ == "__main__":  
    n = 100_000_000  
    inicio = time.time()
```

```
# Executa a contagem pesada duas vezes  
# sequencialmente  
contagem_pesada(n)  
contagem_pesada(n)
```

```
fim = time.time()  
print(f"[Python Sequencial] Tempo: {fim -  
    inicio:.4f} segundos")
```

Esclarecimento de conceito - Gemini

8. Conclusão e Referências:

Este laboratório consolidou conceitos essenciais de programação paralela e memória compartilhada. A prática com Pthreads demonstrou que dividir tarefas entre múltiplas *threads* acelera o processamento, mas o uso de variáveis globais exige controle (muito) rigoroso para evitar condições de corrida. O uso de travas como o *mutex* garante a consistência matemática dos dados, porém deve ser aplicado com moderação para não gerar sobrecargas (*overheads*) que anulem os ganhos de desempenho.

Ficou evidente que a eficiência depende do particionamento adequado do trabalho (estático ou cíclico) para balancear a carga e aproveitar a memória *cache*. Além disso, a escalabilidade possui barreiras físicas e teóricas inegociáveis, como a restrição do barramento de memória em tarefas

memory-bound e a fração sequencial ditada pela Lei de Amdahl. Por fim, constatou-se que o GIL do Python atua como uma trava arquitetural que inviabiliza o paralelismo real em rotinas de processamento intensivo (*CPU-bound*).

<https://www.amd.com/pt/products/processors/laptop/ryzen/7000-series/amd-ryzen-7-7735hs.html>